UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO
Facultad de Ciencias de la Computación · Carrera de Software

Proyecto Fin de Curso, entrega 3
Integración de la base de datos distribuida y del componente de procesamiento
TA-PFC-E3 · Guía para el estudiante y rúbrica de calificación

Aplicaciones Distribuidas (ISR-701) · SPA 2026–2027

Campo	Detalle
Código	TA-PFC-E3
Componente	Trabajo Autónomo (TA)
Modalidad	Grupal, equipos del Proyecto Fin de Curso (PFC) de 3 o 4 integrantes
Carácter	Sumativa
Semana	13 (18 al 24 de enero de 2027)
Cierre	Sábado 23 de enero de 2027, 23:59 (etiqueta de Git `e3` en el repositorio del PFC)
Peso en la nota	1,00 de los 10 puntos del segundo corte
Formativa vinculada	FORM-TL-05 Selección de proveedor de nube (semana 12)
Actividad relacionada	Exposición del 2.º corte (EXP-2), misma semana, en la que el equipo defiende este avance
Unidades que se aplican	Unidad 3: Bases de datos distribuidas (semanas 9 a 11); Unidad 4: Computación paralela y distribuida (semanas 12 y 13)
Docente	Gleiston C. Guerrero-Ulloa, PhD (`gguerrero@uteq.edu.ec`)

1. Propósito de la entrega

En las entregas 1 y 2 el equipo analizó su problema, definió la arquitectura de su sistema y construyó sus microservicios con API REST, API Gateway y contenedores. En esta entrega el sistema deja de guardar sus datos en una base de datos de un solo nodo y pasa a trabajar sobre una **base de datos distribuida**: los datos se reparten entre varios nodos (fragmentación), se copian en más de un nodo (replicación) y el sistema tiene que seguir funcionando, sin perder lo que ya confirmó, cuando un nodo falla. Además, el equipo incorpora un **componente de procesamiento paralelo** sobre los datos del PFC y mide cuánto se acelera al usar más trabajadores.

La entrega aplica los resultados de aprendizaje de las unidades 3 y 4 del sílabo: *implementa soluciones de datos distribuidos (fragmentación, replicación, control de concurrencia y recuperación) evaluando su rendimiento e impacto*, e *implementa soluciones de computación paralela y distribuida con procesamiento concurrente y plataformas en la nube, evaluando su rendimiento.*

La idea central en una frase

El equipo tiene que demostrar con mediciones propias tres cosas: que sus datos están repartidos y copiados con un criterio justificado, que el sistema resiste la caída de un nodo y los accesos simultáneos al mismo dato sin corromperse, y que su componente de procesamiento se acelera al paralelizarlo, en la proporción que la ley de Amdahl permite explicar. En la práctica, cada una de esas tres afirmaciones debe tener en el repositorio un *script* que la reproduce y un archivo con los datos crudos que produjo.

2. Conceptos que se aplican

Esta sección resume, en lenguaje sencillo, los conceptos que la entrega pide aplicar y cómo se ven en el PFC. La bibliografía base para la unidad 3 es [1]-[3].

Concepto	Qué significa y cómo se aplica en el PFC
Repartir los datos (fragmentación)	Dividir una tabla en partes que viven en nodos distintos: por filas (horizontal, por ejemplo por sede o por rango de fechas), por columnas (vertical) o combinando ambas (mixta). Un buen esquema es completo (ningún dato se pierde), reconstruible (se puede volver a armar la tabla original) y disjunto (cada fila está en una sola parte) [1]. En el PFC se elige la clave de fragmentación según las consultas más frecuentes.
Copiar los datos (replicación)	Mantener copias de cada parte en varios nodos. Si la copia se confirma en las réplicas antes de responder al cliente es síncrona; si se confirma después, es asíncrona y puede perder lo más reciente cuando cae el nodo principal [2]. En el PFC se fija el número de réplicas y el modo para cada tipo de dato.
Consistencia, CAP y PACELC	Ante una partición de red, un sistema distribuido elige entre responder siempre (disponibilidad) o responder solo con datos correctos (consistencia) [4]; y aun sin partición, elige entre latencia y consistencia (PACELC) [5]. En el PFC la elección se argumenta por agregado del dominio: por ejemplo, un pago exige consistencia y un catálogo puede tolerar datos algo desactualizados.
Accesos simultáneos (control de concurrencia)	Reglas para que dos transacciones que tocan el mismo dato no lo dejen en un estado inválido: bloqueos (pesimista), comprobación de versión al confirmar (optimista) [6] o aislamiento serializable. En el PFC se demuestra con muchas solicitudes simultáneas sobre el mismo recurso y la comprobación de una regla que nunca debe romperse (invariante).
Recuperación ante fallos	Capacidad de seguir atendiendo y de no perder transacciones confirmadas cuando cae un nodo, y de reincorporarlo después. Las bases de datos SQL distribuidas lo logran replicando con un protocolo de consenso [7]. En el PFC se mide cuánto tarda el sistema en volver a atender y si se perdió algo.
Aceleración (*speedup*) y ley de Amdahl	Si un trabajo tarda T_1 con un trabajador y T_p con p trabajadores, la aceleración es $S(p) = T_1/T_p$ y la eficiencia $E(p) = S(p)/p$. La ley de Amdahl dice que, si solo una fracción f del trabajo es paralelizable, la aceleración máxima es $S(p) = 1/((1-f) + f/p)$ [8]. En el PFC se mide $S(p)$ y se estima qué parte del componente es secuencial.

Concepto	Qué significa y cómo se aplica en el PFC
Modelos de servicio de nube	Qué administra el proveedor y qué el equipo: infraestructura (IaaS), plataforma (PaaS) o funciones (FaaS). En el PFC se elige, a partir de la formativa FORM-TL-05, dónde se desplegaría cada contenedor y la base de datos, y cuánto costaría al mes.

3. Qué debe construir y entregar el equipo

La entrega se organiza en seis módulos. Cada módulo indica entre corchetes el rol responsable sugerido; en equipos de tres integrantes, un mismo estudiante asume dos roles. Todos los integrantes deben tener *commits* propios y sustantivos en la entrega.

Módulo A — Diseño de la distribución de los datos *[Responsable de datos]*

1. Identifique los agregados del dominio del PFC (las entidades que se leen y se escriben juntas) y, para cada uno, las consultas y escrituras más frecuentes.
2. Defina el esquema de fragmentación de al menos **dos tablas principales**: tipo de fragmentación, clave o predicado, número de fragmentos y nodo o región donde vive cada uno. Demuestre que el esquema es completo, reconstruible y disjunto.
3. Defina la replicación: número de réplicas (al menos tres nodos en el clúster), modo síncrono o asíncrono y comportamiento ante la caída del nodo líder.
4. Declare, en una tabla, la posición CAP y PACELC de cada agregado con su justificación.
5. Registre al menos **dos decisiones de arquitectura nuevas** (ADR) en `docs/adr/`: la elección de la tecnología de base de datos distribuida y el esquema de fragmentación. Cada ADR tiene contexto, opciones consideradas, decisión y consecuencias.

Módulo B — Clúster de base de datos distribuida *[Responsable de datos]*

1. Levante un clúster de **al menos tres nodos** con Docker Compose, con verificaciones de salud, volúmenes persistentes y configuración por variables de entorno. La tecnología es de elección del equipo y se justifica en el ADR: por ejemplo, una base de datos SQL distribuida (CockroachDB o YugabyteDB) o PostgreSQL con una extensión de distribución (Citus) o con replicación en *streaming*.
2. Cree el esquema con migraciones versionadas (por ejemplo, Flyway, Liquibase o Alembic) y aplique en ellas la fragmentación del módulo A.
3. Conecte **al menos un microservicio** del PFC al clúster, a través de un balanceador o de una cadena de conexión con varios nodos, de modo que el servicio no dependa de un nodo concreto.
4. Cargue datos de prueba en volumen suficiente para que las mediciones tengan sentido (como mínimo 100 000 registros en la tabla principal), generados con un *script* versionado. Los datos de prueba pueden ser sintéticos, siempre que se declare que lo son y cómo se generaron.

Módulo C — Concurrencia y recuperación ante fallos *[Responsable de calidad]*

1. **Prueba de concurrencia.** Elija una operación crítica del PFC sobre un recurso compartido (por ejemplo, reservar el último cupo o descontar el último ejemplar) y escriba su invariante (por ejemplo, «el stock nunca es negativo»). Lance al menos **100 solicitudes simultáneas** contra el API sobre el mismo recurso y verifique la invariante en la base de datos después de cada corrida. Compare al menos dos mecanismos (por ejemplo, sin control y con control optimista, o nivel de aislamiento por defecto y serializable).
2. **Prueba de fallo.** Con el sistema atendiendo carga continua, detenga un nodo del clúster (`docker stop`) y vuelva a iniciarlo. Mida el tiempo hasta que el sistema vuelve a responder sin errores y compruebe que no se perdió ninguna transacción confirmada antes de la caída (contando las confirmaciones registradas por el cliente contra las filas en la base de datos).
3. Repita cada prueba **al menos cinco veces** y guarde los resultados crudos en CSV.

Módulo D — Componente de procesamiento paralelo *[Responsable de procesamiento]*

1. Implemente un componente que procese datos reales del PFC y que sea útil para el sistema: por ejemplo, un reporte consolidado, un cálculo de indicadores o una detección de anomalías. El componente se ejecuta como servicio o como trabajo independiente dentro del *compose*.
2. Paralelícelo con las herramientas de concurrencia del lenguaje del equipo, de modo que el número de trabajadores p sea un parámetro. Por ejemplo, en Java puede usar `ExecutorService` o *streams* paralelos, y en Python los módulos `multiprocessing` o `concurrent.futures`.
3. Mida el tiempo de ejecución con $p = 1, 2, 4$ y 8 trabajadores (o hasta el número de núcleos de la máquina), con **al menos cinco repeticiones** por configuración después de una corrida de calentamiento que no se cuenta. Registre la máquina usada (procesador, núcleos, memoria y sistema operativo).
4. Calcule $S(p)$ y $E(p)$ con su media, desviación estándar e intervalo de confianza del 95 %. Estime la fracción paralelizable f a partir de los datos y compárela con la métrica de Karp y Flatt, $e = (1/S(p) - 1/p)/(1 - 1/p)$, que estima la fracción secuencial observada [9].
5. Verifique que el resultado del componente es el mismo con cualquier valor de p: paralelizar no puede cambiar la respuesta.

Módulo E — Plan de despliegue en la nube *[Arquitecto]*

1. A partir de la formativa FORM-TL-05, indique el proveedor de nube elegido y, para cada contenedor del *compose* y para la base de datos distribuida, el servicio concreto donde se desplegaría y su modelo (IaaS, PaaS o FaaS).
2. Dibuje el diagrama de despliegue (nodos, regiones o zonas, redes y réplicas).
3. Estime el costo mensual con la calculadora oficial del proveedor, para la carga esperada del PFC, e incluya la captura de la estimación con la fecha. El despliegue real no es obligatorio en esta entrega.

Módulo F — Documento LaTeX de la entrega *[todo el equipo]*

El documento del PFC es acumulativo: la entrega 3 añade un capítulo nuevo al documento de las entregas anteriores, sin borrar lo previo. El capítulo de la entrega 3 tiene este orden:

a) cambios en la arquitectura desde la entrega 2, con el diagrama de contenedores actualizado (modelo C4, nivel 2);

b) diseño de la distribución de los datos (módulo A), con la tabla CAP/PACELC por agregado;

c) implementación del clúster y de las migraciones (módulo B);

d) pruebas de concurrencia y de fallo: método, resultados en tablas y gráficos e interpretación (módulo C);

e) componente de procesamiento: diseño, mediciones, $S(p)$, $E(p)$, estimación de f y discusión de la diferencia con lo ideal (módulo D);

f) plan de despliegue en la nube y costo estimado (módulo E);

g) limitaciones, riesgos abiertos y plan hacia la entrega final;

h) tabla de aportes: qué hizo cada integrante, con los identificadores de sus *commits* principales;

i) declaración de uso de inteligencia artificial.

3.1. Estructura mínima del repositorio

```
/                       README.md, docker-compose.yml, .env.example, .gitignore
/db/                    compose del cluster, migraciones, script de carga de datos
/services/<servicio>/   codigo de cada microservicio (ya existente desde E2)
/processing/            codigo del componente de procesamiento paralelo
/experiments/concurrencia/  script, resultados crudos (CSV) y graficos
/experiments/fallos/        script, resultados crudos (CSV) y graficos
/experiments/speedup/       script, resultados crudos (CSV), maquina.txt y graficos
/docs/                  documento .tex del PFC, .bib, figuras y PDF compilado
/docs/adr/              decisiones de arquitectura (ADR)
```

El `README.md` explica cómo levantar el clúster y el sistema, cómo cargar los datos, cómo reproducir cada experimento y cómo compilar el documento (compilador, archivo principal, orden de comandos y dependencias). El archivo `.env` con valores reales no se sube.

3.2. Cómo se entrega

1. Todo el trabajo está en el repositorio público del PFC, con la etiqueta `e3` creada y enviada hasta el sábado 23 de enero de 2027 a las 23:59:

```
git tag -a e3 -m "PFC entrega 3: BDD distribuida y componente de procesamiento"
git push origin e3
```

2. En la tarea del SGA, cada integrante sube **solamente un documento PDF de UNA SOLA página** con los datos identificativos (universidad, facultad, carrera, asignatura, código y nombre de la actividad, integrantes del equipo, paralelo y fecha) y la URL del repositorio escrita completa en una sola línea. El documento de la entrega, el código y los resultados van en el repositorio, no en el SGA.

4. Evidencias de autoría

1. **Historial continuo.** El trabajo se construye a lo largo de las semanas, con *commits* frecuentes y mensajes descriptivos escritos por cada autor. Una entrega subida en uno o pocos *commits* masivos al final es un indicio de autoría no propia (I6 del encuadre).

2. **Resultados que se regeneran.** Cada cifra del documento sale de un CSV de `experiments/` y se puede regenerar con su *script*. Los resultados escritos a mano, o producidos por un programa que los inventa (por ejemplo, con números aleatorios en lugar de medir), se consideran fabricados.
3. **Capturas propias.** Las capturas de las pruebas muestran el usuario del sistema en el *prompt* y la fecha y hora de la máquina.
4. **Defensa.** En la exposición del 2.º corte, cualquier integrante debe poder explicar y modificar cualquier parte de esta entrega.

5. Criterios de piso

Su incumplimiento implica calificación CERO en la entrega

P1. PDF de una sola página con carátula y URL. Cada estudiante sube al SGA solamente un documento PDF de UNA SOLA página con los datos identificativos y, en una sola línea, la URL del repositorio donde está todo el trabajo. Si el PDF tiene más de una página, si se suben otros archivos en lugar de este PDF, si la URL no cabe en una sola línea o está rota, o si el repositorio no es accesible (privado o inexistente), la calificación es CERO.

P2. PDF reproducible. Si el PDF del documento no se genera correctamente al clonar el repositorio en la etiqueta `e3` y compilar el archivo `.tex`, la calificación es CERO. Las instrucciones de compilación (compilador, orden de comandos, archivo principal y dependencias) deben estar en el `README.md`.

P3. Plazo. La etiqueta `e3` no existe o apunta a contenido subido después del sábado 23 de enero de 2027 a las 23:59.

P4. Integridad. Plagio, contenido generado por IA generativa presentado como propio, mediciones, capturas o resultados fabricados, o **una sola** referencia inexistente o cuyo DOI lleve a una obra con datos diferentes a los citados [10].

P5. Formativa vinculada. El equipo que no entregó la formativa FORM-TL-05 no puede presentar esta entrega y obtiene CERO en ella, salvo justificación aprobada.

6. Cómo se calcula la nota

Como en toda actividad grupal de la asignatura, la nota es individual:

$$\text{Nota individual} = 0{,}60 \times \text{Nota del equipo} + 0{,}40 \times \text{Nota del aporte individual}$$

Cada indicador se califica con un nivel de 0 a 4 según su calidad, no por su sola presencia: el nivel es el que corresponde al aspecto de calidad peor cumplido (veracidad, precisión técnica, profundidad, pertinencia, suficiencia de la evidencia, coherencia, autoría, reproducibilidad y calidad de las fuentes). Un indicador ausente, no verificable en el repositorio o falso vale 0.

$$\text{Nota previa} = \sum_i \frac{w_i \times n_i}{4} \times \frac{10}{100}, \qquad \text{Nota final} = \text{máx}(0,\ \text{Nota previa} - \text{Penalizaciones})$$

donde w_i es el peso del indicador en porcentaje y n_i su nivel. En la retroalimentación se informan siempre la nota previa y la nota final. La nota aporta al segundo corte con su peso: Nota/10 × 1,00.

7. Rúbrica del equipo

Los pesos de las dimensiones son los del encuadre para las entregas del PFC (D1 15, D2 15, D3 30, D4 20, D5 10, D6 10). La tabla 1 reparte cada dimensión entre los indicadores de esta entrega y la tabla 2 da sus descriptores de nivel.

Tabla 1: Indicadores de la nota del equipo

Ind.	Indicador	Módulo y evidencia	Peso
D1 Cumplimiento y completitud (15)			
1.1	Entregables completos y reproducibles	Estructura del repositorio, README, etiqueta `e3`, capítulo con sus nueve partes	15
D2 Fundamentación conceptual (15)			
2.1	Diseño de la distribución de los datos	Módulo A: fragmentación con sus tres propiedades, replicación y tabla CAP/PACELC por agregado	8
2.2	Decisiones de arquitectura (ADR)	Módulo A: dos ADR nuevos con opciones comparadas	7
D3 Desarrollo técnico y evidencia (30)			
3.1	Clúster y fragmentación implementados	Módulo B: tres nodos o más, migraciones, datos cargados, microservicio conectado	8
3.2	Replicación y recuperación ante fallos	Módulo C: prueba de fallo con cinco repeticiones y CSV	7
3.3	Control de concurrencia	Módulo C: 100 solicitudes simultáneas, invariante y dos mecanismos	7
3.4	Componente de procesamiento paralelo	Módulo D: componente útil, p parametrizable, resultado idéntico para todo p	8
D4 Análisis crítico y resultados (20)			
4.1	Análisis de fallos y de concurrencia	Módulo C: interpretación de tiempos de recuperación, pérdidas y violaciones de la invariante	7
4.2	Análisis de la aceleración	Módulo D: $S(p)$, $E(p)$ con dispersión, f estimada, Karp y Flatt y explicación de la diferencia con lo ideal	8
4.3	Decisión de nube y costo	Módulo E: servicio por componente, diagrama de despliegue y costo con captura	5
D5 Comunicación (10)			
5.1	Documento LaTeX	Capítulo claro, sin desbordes ni referencias sin resolver, tablas y gráficos legibles	10
D6 Referencias, ética e integridad (10)			
6.1	Referencias y declaración de IA	Fuentes IEEE recientes y verificables citadas donde se usan; declaración de IA	10
	Total		**100**

Tabla 2: Rúbrica del equipo: descriptores de nivel por indicador

Indicador (peso)	4 Excelente	3 Bueno	2 Aceptable	1 Insuficiente	0 Ausente
1.1 Entregables (15)	Todo lo de la sección 3 está en la etiqueta `e3`; el sistema, los experimentos y el documento se reproducen desde un clon limpio siguiendo el README.	Falta un elemento menor o un paso del README requiere un ajuste menor.	Faltan varios elementos o un experimento no se puede reproducir.	Solo una parte mínima de lo pedido.	No hay entrega o no corresponde a la E3.
2.1 Distribución de datos (8)	Dos tablas o más fragmentadas con criterio derivado de las consultas reales; se demuestran completitud, reconstrucción y disyunción; replicación definida; posición CAP/PACELC argumentada por agregado.	Diseño correcto con una propiedad sin demostrar o un agregado sin justificar.	Una sola tabla fragmentada, o posición CAP afirmada para todo el sistema.	Confunde fragmentación con replicación o el criterio no se relaciona con el PFC.	Sin diseño de distribución.
2.2 ADR (7)	Dos ADR o más, cada uno con contexto, al menos dos opciones comparadas con criterios, decisión y consecuencias negativas reconocidas.	ADR completos con una comparación superficial.	ADR sin opciones alternativas o sin consecuencias.	Un solo ADR o ADR que solo describen.	Sin ADR.
3.1 Clúster (8)	Clúster de tres nodos o más con salud, volúmenes y `.env`; migraciones versionadas aplican la fragmentación; 100 000 registros o más cargados por *script*; un microservicio conectado sin depender de un nodo.	Funciona con un detalle menor (por ejemplo, sin verificación de salud).	Clúster de menos de tres nodos, o fragmentación no aplicada en el esquema real.	Clúster que no se levanta sin pasos manuales o ningún servicio conectado.	Sin clúster.
3.2 Fallos (7)	Caída y reincorporación de un nodo bajo carga, cinco repeticiones o más, tiempo de recuperación medido y conteo de transacciones confirmadas contra filas persistidas, todo en CSV.	Prueba completa con menos repeticiones o un dato sin registrar.	Prueba ejecutada una sola vez o sin carga continua.	Solo se describe la prueba, sin ejecutarla.	Sin prueba de fallo.

Indicador (peso)	4 Excelente	3 Bueno	2 Aceptable	1 Insuficiente	0 Ausente
3.3 Concurrencia (7)	100 solicitudes simultáneas o más sobre el mismo recurso, invariante comprobada en la base de datos tras cada corrida, dos mecanismos comparados, cinco repeticiones.	Completa con un solo mecanismo bien probado o menos repeticiones.	Menos de 100 solicitudes o la invariante no se verifica en la base de datos.	Solo pruebas secuenciales.	Sin prueba de concurrencia.
3.4 Procesamiento (8)	Componente útil para el PFC sobre sus datos, p parametrizable, mediciones con calentamiento y cinco repeticiones por p, máquina registrada, mismo resultado para todo p.	Completo con un detalle menor del protocolo de medición.	Componente artificial (por ejemplo, un bucle sin relación con el PFC) o menos de tres valores de p.	No paraleliza realmente o no se mide.	Sin componente.
4.1 Análisis de fallos y concurrencia (7)	Interpreta tiempos de recuperación y pérdidas con su dispersión; explica por qué cada mecanismo viola o no la invariante y su costo en latencia o reintentos; reconoce limitaciones.	Análisis correcto con una conclusión sin sustento suficiente.	Describe resultados sin interpretarlos.	Conclusiones genéricas que no salen de los datos.	Sin análisis.
4.2 Análisis de aceleración (8)	Gráficos de $S(p)$ y $E(p)$ con media e intervalo de confianza; f estimada y contrastada con Karp y Flatt; explica la diferencia con lo ideal (fracción secuencial, sincronización, E/S o memoria).	Correcto con alguna imprecisión o sin intervalo de confianza.	Calcula $S(p)$ sin estimar f o sin explicar la diferencia.	Cifras sin método o con errores de cálculo.	Sin análisis.
4.3 Nube y costo (5)	Servicio y modelo para cada componente, diagrama de despliegue con zonas y réplicas, costo mensual con captura fechada de la calculadora oficial y justificado con la carga esperada.	Completo con un componente sin asignar.	Sin diagrama o sin captura de la estimación.	Elección sin justificación ni costo.	Sin plan de nube.

Indicador (peso)	4 Excelente	3 Bueno	2 Aceptable	1 Insuficiente	0 Ausente
5.1 Documento (10)	Capítulo completo, claro y en el orden pedido; sin desbordes ni referencias sin resolver; tablas `booktabs` y gráficos con ejes y unidades.	Defectos menores de formato o redacción.	Varios defectos que dificultan la lectura.	Documento desordenado o incompleto.	Sin documento legible.
6.1 Referencias e integridad (10)	Referencias IEEE recientes y verificables, todas citadas donde se usan; declaración de uso de IA completa; aportes de cada integrante visibles en el historial.	Reales y correctas con algún campo omitido.	Reales pero escasas, antiguas o de baja calidad.	Reales pero mal citadas, o sin declaración de IA.	Sin referencias o en formato distinto de IEEE.

Nota sobre 6.1: una referencia inexistente, o cuyo DOI lleve a una obra con datos diferentes a los citados, no se califica aquí: implica calificación CERO en la entrega (criterio de piso P4).

8. Rúbrica del aporte individual

El aporte individual se califica con el historial del repositorio hasta la etiqueta `e3` (unificando las distintas identidades de Git que use cada estudiante) y con la tabla de aportes del documento.

Tabla 3: Criterios del aporte individual

Criterio	Nivel 4 (Excelente)	Peso
A1 Amplitud e importancia	Sus *commits* cubren uno o más módulos completos de la entrega y son necesarios para que el sistema funcione o para que los resultados existan.	30
A2 Complejidad técnica	Su aporte incluye trabajo técnico no trivial (configuración del clúster, pruebas de concurrencia o de fallo, paralelización, medición), no solo formato o documentación.	25
A3 Autoría en el documento	Redactó secciones del capítulo que corresponden a lo que construyó, con aportes verificables en el historial del `.tex`.	20
A4 Continuidad del trabajo	Sus *commits* se distribuyen a lo largo de las semanas 10 a 13, con mensajes descriptivos propios, sin cargas masivas al final.	25
Total		**100**

Los niveles 3, 2, 1 y 0 siguen la escala general del encuadre: 3 cubre al menos el 75 % con deficiencias menores; 2 entre el 50 y el 74 %; 1 menos del 50 %; 0 ausente o no verificable. Un estudiante sin *commits* propios en la entrega obtiene 0 en el aporte individual.

9. Penalizaciones que no anulan la nota

Situación detectada	Efecto (sobre 10)
Falta la declaración de uso de inteligencia artificial	$-0{,}3$
Referencias en formato APA u otro distinto de IEEE	$6.1 = 0$
Dos indicios de uso de IA generativa o de autoría no propia, I1 e I3 a I8 del encuadre (nivel B)	$-1{,}5$
Tres o más indicios, o resultados no verificados presentados como verificados (nivel C)	$-3{,}0$

10. Calendario sugerido

Semana	Qué debería estar terminado
10 (28 dic – 3 ene)	Módulo A: agregados, esquema de fragmentación y replicación, ADR en borrador.
11 (4 – 10 ene)	Módulo B: clúster levantado, migraciones y datos cargados; microservicio conectado. Primeras corridas del módulo C.
12 (11 – 17 ene)	Módulo C completo con sus repeticiones; módulo D implementado; formativa FORM-TL-05 entregada.
13 (18 – 23 ene)	Mediciones del módulo D, módulo E, capítulo del documento, exposición EXP-2 y etiqueta `e3` hasta el sábado a las 23:59.

Lista de verificación antes de crear la etiqueta

- ☐ Desde un clon limpio, el clúster y el sistema se levantan con las instrucciones del `README.md`.
- ☐ Cada experimento se reproduce con su *script* y regenera su CSV.
- ☐ Cada cifra del documento coincide con un CSV de `experiments/`.
- ☐ El componente de procesamiento da el mismo resultado con cualquier p.
- ☐ El PDF del documento se genera desde el clon limpio y no tiene `??` ni desbordes.
- ☐ Cada referencia tiene su DOI o ISBN comprobado contra la obra real.
- ☐ La tabla de aportes indica qué hizo cada integrante y sus *commits*.
- ☐ El capítulo termina con la declaración de uso de IA.

11. Retroalimentación y revisión

El docente publica en el SGA la calificación con la retroalimentación individual de cada integrante, que incluye la nota del equipo, la nota del aporte individual, la nota previa y la nota final. Ante un desacuerdo, el estudiante lo plantea primero al docente, que responde en tres días; si persiste, acude a la Coordinación de Carrera.

Referencias

[1] M. T. Özsu y P. Valduriez, *Principles of Distributed Database Systems*, 4.ª ed. Cham, Switzerland: Springer, 2020, ISBN: 978-3-030-26252-5. DOI: 10.1007/978-3-030-26253-2.

[2] M. Kleppmann y C. Riccomini, *Designing Data-Intensive Applications*, 2.ª ed. Sebastopol, CA, USA: O'Reilly Media, 2026, ISBN: 978-1-098-11906-5.

[3] M. van Steen y A. S. Tanenbaum, *Distributed Systems*, 4.ª ed. Amsterdam, The Netherlands: Maarten van Steen, 2023, ISBN: 978-90-815406-3-6. dirección: https://www.distributed-systems.net/.

[4] S. Gilbert y N. Lynch, «Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services,» *ACM SIGACT News*, vol. 33, n.º 2, págs. 51-59, 2002. DOI: 10.1145/564585.564601.

[5] D. Abadi, «Consistency Tradeoffs in Modern Distributed Database System Design: CAP is Only Part of the Story,» *Computer*, vol. 45, n.º 2, págs. 37-42, 2012. DOI: 10.1109/MC.2012.33.

[6] H. T. Kung y J. T. Robinson, «On Optimistic Methods for Concurrency Control,» *ACM Transactions on Database Systems*, vol. 6, n.º 2, págs. 213-226, 1981. DOI: 10.1145/319566.319567.

[7] R. Taft, I. Sharif, A. Matei et al., «CockroachDB: The Resilient Geo-Distributed SQL Database,» en *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, Portland, OR, USA: ACM, 2020, págs. 1493-1509. DOI: 10.1145/3318464.3386134.

[8] G. M. Amdahl, «Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities,» en *Proceedings of the April 18–20, 1967, Spring Joint Computer Conference (AFIPS '67 Spring)*, Atlantic City, NJ, USA: ACM Press, 1967, págs. 483-485. DOI: 10.1145/1465482.1465560.

[9] A. H. Karp y H. P. Flatt, «Measuring Parallel Processor Performance,» *Communications of the ACM*, vol. 33, n.º 5, págs. 539-543, 1990. DOI: 10.1145/78607.78614.

[10] Association for Computing Machinery, *ACM Code of Ethics and Professional Conduct*, 2018. dirección: https://www.acm.org/code-of-ethics (visitado 26-10-2026).